

## Ezora AI Voice Agent

An intelligent, stateful AI voice agent built with LangGraph to automate customer conversations, qualify leads, and perform real-time call analytics. The system maintains conversational context, intelligently routes queries using **RAG**, and extracts vital business metrics dynamically.

### Features

**Stateful Conversations:** Turn-by-turn state management utilizing LangGraph.

**Dynamic Intent Routing:** Automatically classifies user intent to route to product discussions, support, or human handoff.

**RAG Integration:** Retrieves company knowledge and product information dynamically from ingested documents (source.md, source.pdf).

**Automated Lead Qualification:** Extracts **CRM** usage, industry, call volume, and pain points, generating a real-time Lead Score.

**Call Analytics:** Post-call evaluation of overall customer sentiment and call outcomes (e.g., qualified lead, demo requested).

**Real-Time WebSocket API:** Production-ready FastAPI implementation allowing real-time streaming text for voice synthesis integration.

**Interactive CLI:** Test the agent locally via a conversational command-line interface.

### Architecture

The agent is structured as a directed state graph (AgentState) where each turn passes through specific nodes:

**Intent Router (intent\_router\_node.py):** Classifies the incoming message, rewrites queries for better context, and decides if external knowledge is needed.

**RAG Node (rag\_node.py):** Fetches relevant context from the company knowledge base if the intent demands specific product or company knowledge.

**Conversation Node (conversation\_node.py):** Generates natural, concise responses optimized for voice synthesis (strict under 80 words limit).

**Lead Qualification (lead\_qualification\_node.py):** Dynamically updates the AgentState with extracted business data and scores the lead out of **100**.

**Analytics Node (analytics\_node.py):** Triggers upon call completion to synthesize the final outcome and customer sentiment.

### Tech Stack

**Core Frameworks:** LangGraph, LangChain, OpenAI (gpt-4o-mini)

Backend **API**: FastAPI, WebSockets

Package Management: Poetry (pyproject.toml)

Database: PostgreSQL (for storing analytics via SQLAlchemy)

Data Generation: Faker (for mock analytics data)

## Setup and Installation

This project requires Python 3.11+ and uses Poetry for dependency management.

## Clone the repository

```
git clone cd ezora-voice-agent
```

## Install dependencies

```
poetry install
```

## Environment Setup

Create a .env file in the root directory and add your database configuration and OpenAI **API** key:

```
OPENAI_API_KEY=your_openai_api_key_here
```

```
DATABASE_URL=postgresql://user:password@localhost:5432/ezora
```

## Usage

### Running the WebSocket Server (Production/**API**)

Start the main FastAPI application server which exposes the /ws/chat endpoint:

```
poetry run uvicorn main:app --reload
```

### Interactive **CLI** Testing

Simulate a conversation turn-by-turn in your terminal to observe intent classification, **RAG** retrieval, and state changes:

```
poetry run python cli_chat.py
```